

PRACTICAL WEEK-6

1. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define FIFO_NAME "server_fifo"
#define BUFFER_SIZE 256

int main()
{
    char buffer[BUFFER_SIZE];

    // Create named pipe
    if (mkfifo(FIFO_NAME, 0666) == -1)
    {
        perror("mkfifo");
    }

    printf("Server started.\n");
    printf("Waiting for client messages...\n");

    while (1)
    {
        int fd = open(FIFO_NAME, O_RDONLY);

        if (fd == -1)
        {
            perror("open");
            return 1;
        }

        int bytes_read;

        while ((bytes_read = read(fd, buffer, BUFFER_SIZE - 1)) > 0)
        {
            while ((bytes_read = read(fd, buffer, BUFFER_SIZE - 1)) > 0)
            {
                buffer[bytes_read] = '\0';

                printf("Server received: %s", buffer);

                printf("Server response: Message processed successfully.\n");
            }

            close(fd);
        }

        return 0;
    }
}
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define FIFO_NAME "server_fifo"
#define BUFFER_SIZE 256

int main()
{
    char message[BUFFER_SIZE];

    printf("Enter message for server: ");
    fgets(message, BUFFER_SIZE, stdin);

    int fd = open(FIFO_NAME, O_WRONLY);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    write(fd, message, strlen(message));

    close(fd);

    printf("Message sent to server successfully.\n");

    return 0;
}

```

```

laasya@laasya:~$ cd session6_fifo
-bash: cd: session6_fifo: No such file or directory
laasya@laasya:~$ ./client
Enter message for server: HELLO Server
Message sent to server successfully.
laasya@laasya:~$ |

```

```

-bash: cd: session6_fifo: No such file or directory
laasya@laasya:~$ nano server.c
laasya@laasya:~$ nano client.c
laasya@laasya:~$ gcc server.c -o server
laasya@laasya:~$ gcc client.c -o client
laasya@laasya:~$ ./server
Server started.
Waiting for client messages...
Server received: HELLO Server
Server response: Message processed successfully.

```

2. Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```

#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int signal)
{
    if (signal == SIGINT)
    {
        printf("\nSIGINT received.\n");
        printf("Signal handler executed for SIGINT.\n");
    }
    else if (signal == SIGTERM)
    {
        printf("\nSIGTERM received.\n");
        printf("Program terminating safely.\n");
        exit(0);
    }
    else if (signal == SIGUSR1)
    {
        printf("\nSIGUSR1 received.\n");
        printf("User-defined signal handler executed.\n");
    }
}

int main()
{
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Signal handling program started.\n");
    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1)
    {

```

```

        while (1)
        {
            printf("Program is running...\n");
            sleep(3);
        }

        return 0;
    }
}

```

```
laasya@laasya:~$ nano signals.c
laasya@laasya:~$ gcc signals.c -o signals
laasya@laasya:~$ ./signals
Signal handling program started.
Process ID: 591
Waiting for signals...
Program is running...
Program is running...
Program is running...
Program is running...
|
```

```
laasya@laasya: ~ × + ▾
laasya@laasya:~$ kill -SIGUSR1 4521
-bash: kill: (4521) - No such process
laasya@laasya:~$ kill -SIGINT 4521
-bash: kill: (4521) - No such process
laasya@laasya:~$ |
```